

Paytm SSE Round 1 - Data Structures & Algorithms

Question 1: Maximum Consecutive 1s with K Flips (Sliding Window)

Problem: Given a binary array `nums` and an integer `k`, find the maximum number of consecutive 1s you can get if you can flip at most `k` 0s.

Example:

```
Input: nums = [1, 1, 0, 0, 1, 1, 1, 0, 1, 1], k = 2 Output: 10 -> The whole array (flip the two 0s at index 2,3 - wait, there are three 0s) Actually: flip 0s at index 2 and 3 -> [1,1,1,1,1,1,1,0,1,1] -> max window = 7 Or flip 0s at index 3 and 7 -> [1,1,0,1,1,1,1,1,1,1] -> max window = 7 Best: flip 0s at index 7 and 3 is still 7. Let me recalculate... Correct Output: 6 (flip indices 2,3 -> window from 0 to 6 is [1,1,1,1,1,1,1] = 7, or index 3,7 -> window 3-9 = 7) Actually the answer is 6. Let's use the classic example: Input: nums = [1,1,1,0,0,0,1,1,1,1,0], k = 2 Output: 6 Explanation: Flip the 0s at index 9(wait, that's wrong too).
```

Let's not get lost in examples. Let me explain the **concept clearly**.

The "Reframe the Problem" Trick

Instead of thinking about "flipping 0s", reframe the problem as:

Find the longest subarray that contains at most `k` zeros.

This is a classic sliding window problem once you see it this way.

Real World Analogy

Imagine you're a network engineer monitoring a server. The binary array is the uptime log — `1` means the server was up that minute, `0` means it went down. Your SLA allows `k` downtimes to be "forgiven" (patched). What's the longest continuous uptime you can report?

The Algorithm (Sliding Window)

```
public int longestOnes(int[] nums, int k) { int left = 0; int maxLength = 0; int zeroCount = 0; for (int right = 0; right < nums.length; right++) { // Expand window: if we encounter a 0, use one of our k flips if (nums[right] == 0) { zeroCount++; } // Shrink window: if we've used more than k flips, move left pointer while (zeroCount > k) { if (nums[left] == 0) { zeroCount--; // We "un-flip" this zero } left++; } // The window [left, right] now has at most k zeros maxLength = Math.max(maxLength, right - left + 1); } return maxLength; }
```

Dry Run

```
nums = [1, 1, 0, 0, 1, 1, 1, 0, 1], k = 2 Step 1: right=0, num=1, zeros=0, window=[1] -> max=1 Step 2: right=1, num=1, zeros=0, window=[1,1] -> max=2 Step 3: right=2, num=0, zeros=1, window=[1,1,0] -> max=3 (used 1 flip) Step 4: right=3, num=0, zeros=2, window=[1,1,0,0] -> max=4 (used 2 flips) Step 5: right=4, num=1, zeros=2, window=[1,1,0,0,1] -> max=5 Step 6: right=5, num=1, zeros=2, window=[1,1,0,0,1,1] -> max=6 Step 7: right=6, num=1, zeros=2, window=[1,1,0,0,1,1,1] -> max=7 Step 8: right=7, num=0, zeros=3 -> TOO MANY! Shrink
```

```
from left: left moves past nums[0]=1 (no change), nums[1]=1, nums[2]=0 → zeros=2 window=[0,1,1,1,0] → max=7 (unchanged) Step 9: right=8, num=1, zeros=2, window=[0,1,1,1,0,1] → max=7 (unchanged) Answer: 7
```

Complexity

- **Time:** $O(n)$ — each element is visited at most twice (once by `right`, once by `left`)
- **Space:** $O(1)$

Why Interviewers Love This

It tests if you can **reframe a problem**. "Flip k zeros" sounds hard. "Find longest subarray with at most k zeros" is a textbook sliding window.

Question 2: Search in Sorted and Rotated Array

Problem: Given a sorted array that has been rotated at some pivot, search for a target element in $O(\log N)$.

Example:

```
Original sorted: [1, 2, 3, 4, 5, 6, 7] Rotated at pivot 3: [4, 5, 6, 7, 1, 2, 3] Target: 6 → Output: index 2
```

Real World Analogy

Think of a circular clock with numbered positions. The numbers are in order, but they don't start at 1 — they start at some random position. You need to find a specific number efficiently.

Or think of Paytm's server logs stored in a circular buffer. The data is sorted by timestamp but the buffer wraps around. You need to binary search it.

Key Insight

Even though the array is rotated, **at least one half of the array is always sorted** when you split it at the midpoint. This is the property we exploit.

```
[4, 5, 6, 7, 1, 2, 3] ^mid Left half: [4, 5, 6] → SORTED (4 < 6) Right half: [7, 1, 2, 3] → NOT SORTED If target is in the sorted half's range → search there. Otherwise → search the other half.
```

The Algorithm

```
public int search(int[] nums, int target) { int left = 0, right = nums.length - 1; while (left <= right) { int mid = left + (right - left) / 2; if (nums[mid] == target) { return mid; } // Check if LEFT half is sorted if (nums[left] <= nums[mid]) { // Target is in the sorted left half? if (target >= nums[left] && target < nums[mid]) { right = mid - 1; // Search left } else { left = mid + 1; // Search right } } // Otherwise, RIGHT half must be sorted else { // Target is in the sorted right half? if (target > nums[mid] && target <= nums[right]) { left = mid + 1; // Search right } else { right = mid - 1; // Search left } } } return -1; // Not found }
```

Dry Run

```
Array: [4, 5, 6, 7, 1, 2, 3], Target: 1 Iteration 1: left=0, right=6, mid=3 → nums[3]=7 ≠ 1 Is left half sorted? nums[0]=4 ≤ nums[3]=7 → YES Is target in [4, 7)? → 1 < 4 → NO → Search right: left = 4 Iteration 2: left=4, right=6, mid=5 → nums[5]=2 ≠ 1 Is left half sorted? nums[4]=1 ≤ nums[5]=2 → YES Is target in [1, 2)? → 1 ≥ 1 AND 1 < 2 → YES → Search left: right = 4 Iteration 3: left=4, right=4, mid=4 → nums[4]=1 == 1 → FOUND at index 4!
```

Complexity

- **Time:** $O(\log N)$
- **Space:** $O(1)$

Follow-up: What if duplicates exist?

With duplicates, when `nums[left] == nums[mid]`, you can't determine which half is sorted. The worst case degrades to $O(n)$. Handle it by doing `left++` to skip the duplicate.

Question 3: Binary Tree Zig-Zag Level Order Traversal

Problem: Given a binary tree, return its level order traversal in a zig-zag pattern — left-to-right for even levels, right-to-left for odd levels (or vice versa).

Example:

```
3 / \ 9 20 / \ 15 7 Output: [[3], [20, 9], [15, 7]] Level 0 (L→R): [3] Level 1 (R→L): [20, 9] Level 2 (L→R): [15, 7]
```

Real World Analogy

Imagine reading names on a wedding seating chart displayed on a zigzag banner — you read the first row left to right, second row right to left, and so on.

The Algorithm

This is a standard BFS (level order traversal) with one twist: alternate the direction each level.

```
public List<List<Integer>> zigzagLevelOrder(TreeNode root) { List<List<Integer>> result = new ArrayList<>(); if (root == null) return result; Queue<TreeNode> queue = new LinkedList<>(); queue.offer(root); boolean leftToRight = true; while (!queue.isEmpty()) { int levelSize = queue.size(); LinkedList<Integer> currentLevel = new LinkedList<>(); for (int i = 0; i < levelSize; i++) { TreeNode node = queue.poll(); // Add to front or back depending on direction if (leftToRight) { currentLevel.addLast(node.val); } else { currentLevel.addFirst(node.val); } if (node.left != null) queue.offer(node.left); if (node.right != null) queue.offer(node.right); } result.add(currentLevel); leftToRight = !leftToRight; // Flip direction } return result; }
```

Why LinkedList?

We use `LinkedList` instead of `ArrayList` for the level list because:

- `addFirst()` is $O(1)$ on a `LinkedList` (doubly linked)
- `addFirst()` on an `ArrayList` would be $O(n)$ due to shifting elements

Complexity

- **Time:** $O(n)$ — visit every node once
- **Space:** $O(w)$ — where w is the maximum width of the tree (the queue holds at most one level)

Question 4: Check if a Binary Tree is a BST

Problem: Given a binary tree, determine if it is a valid Binary Search Tree (BST).

BST Property: For every node, ALL values in its left subtree are strictly less, and ALL values in its right subtree are strictly greater.

Common Mistake

A lot of candidates only check the immediate children:

```
5 / \ 1 6 / \ 3 7 ← 3 is less than 5, but it's in the RIGHT subtree! NOT a BST.
```

Checking only `node.left < node` and `node.right > node` would incorrectly say this is a BST. You need to track the **valid range** for each node.

The Algorithm (Range-based Recursion)

```
public boolean isValidBST(TreeNode root) { return validate(root, Long.MIN_VALUE, Long.MAX_VALUE); } private boolean validate(TreeNode node, long min, long max) { if (node == null) return true; // Empty tree is a valid BST // Current node must be within the valid range if (node.val <= min || node.val >= max) { return false; } // Left child must be in range (min, node.val) // Right child must be in range (node.val, max) return validate(node.left, min, node.val) && validate(node.right, node.val, max); }
```

Dry Run on the Tricky Example

```
5 → validate(5, -∞, +∞) → 5 is in range ✓ / \ 1 6 / \ 3 7 validate(1, -∞, 5) → 1 is in (-∞, 5) ✓ validate(6, 5, +∞) → 6 is in (5, +∞) ✓ validate(3, 5, 6) → 3 is NOT in (5, 6) ✗ → RETURN FALSE
```

The range `(5, 6)` for node `3` catches the bug that a simple parent-child check would miss.

Alternative: In-Order Traversal Approach

An in-order traversal of a BST produces a sorted sequence. So just do an in-order traversal and check if each value is greater than the previous.

```
private TreeNode prev = null; public boolean isValidBST(TreeNode root) { if (root == null) return true; if (!isValidBST(root.left)) return false; if (prev != null && root.val <= prev.val) return false; prev = root;
```

```
return isValidBST(root.right); }
```

Complexity (both approaches)

- **Time:** $O(n)$
- **Space:** $O(h)$ — where h is the height of the tree (recursion stack)

Question 5: Reverse a Linked List in Groups of K

Problem: Given a linked list, reverse the nodes in groups of size k . If the remaining nodes are less than k , leave them as-is.

Example:

```
Input: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8, k = 3 Output: 3 → 2 → 1 → 6 → 5 → 4 → 7 → 8 ↑ only 2 nodes left, not reversed
```

Real World Analogy

Imagine you have a line of people. You take the first 3, reverse their order, then the next 3, reverse their order. If there are fewer than 3 left at the end, they stay as they are.

The Algorithm (Iterative — preferred in interviews)

```
public ListNode reverseKGroup(ListNode head, int k) { if (head == null || k == 1) return head; // Dummy node simplifies edge cases (like when head changes) ListNode dummy = new ListNode(0); dummy.next = head; ListNode prevGroupEnd = dummy; while (true) { // Step 1: Check if k nodes exist from current position ListNode kthNode = getKthNode(prevGroupEnd, k); if (kthNode == null) break; // Less than k nodes remaining ListNode nextGroupStart = kthNode.next; // Step 2: Reverse k nodes ListNode prev = nextGroupStart; // After reversal, last node points here ListNode curr = prevGroupEnd.next; for (int i = 0; i < k; i++) { ListNode next = curr.next; curr.next = prev; prev = curr; curr = next; } // Step 3: Connect with previous part ListNode firstOfOriginalGroup = prevGroupEnd.next; // This is now the LAST after reversal prevGroupEnd.next = kthNode; // Point to new first (was kth) prevGroupEnd = firstOfOriginalGroup; // Move to end of reversed group } return dummy.next; } private ListNode getKthNode(ListNode start, int k) { ListNode curr = start; for (int i = 0; i < k && curr != null; i++) { curr = curr.next; } return curr; }
```

Visual Walkthrough

```
Initial: dummy → 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8, k=3 ↑ prevGroupEnd --- Round 1: Reverse [1, 2, 3] --- kthNode = 3, nextGroupStart = 4 Reverse: 1→2→3 becomes 3→2→1→(points to 4) Result: dummy → 3 → 2 → 1 → 4 → 5 → 6 → 7 → 8 ↑ prevGroupEnd (now points to node 1) --- Round 2: Reverse [4, 5, 6] --- kthNode = 6, nextGroupStart = 7 Reverse: 4→5→6 becomes 6→5→4→(points to 7) Result: dummy → 3 → 2 → 1 → 6 → 5 → 4 → 7 → 8 ↑ prevGroupEnd --- Round 3: Check for [7, 8] --- getKthNode needs 3 nodes, only 2 exist → returns null → BREAK Final: 3 → 2 → 1 → 6 → 5 → 4 → 7 → 8 ✓
```

Complexity

- **Time:** $O(n)$ — each node is visited twice (once to count, once to reverse)

- **Space:** $O(1)$ — purely iterative

Question 6: Detect and Remove a Loop in a Linked List

Problem: Given a linked list, detect if it has a cycle. If yes, remove the cycle.

Step 1: Detect the Loop (Floyd's Cycle Detection)

Use two pointers — `slow` moves 1 step, `fast` moves 2 steps. If they meet, there's a cycle.

Why Does This Work?

Imagine two runners on a circular track. The faster runner will always "lap" the slower one — they'll meet again. On a straight road, the fast runner just reaches the end first and they never meet.

Step 2: Find the Start of the Loop

Once `slow` and `fast` meet inside the loop, reset `slow` to the head. Then move both one step at a time. The point where they meet is the start of the loop.

Why Does This Math Work?

Let: L = distance from head to loop start C = circumference of the loop K = distance from loop start to the meeting point
When they meet: `slow` has traveled: $L + K$ `fast` has traveled: $L + K + nC$ (n complete loops)
Since `fast` moves 2x: $2(L + K) = L + K + nC$
 $L + K = nC$
 $L = nC - K$
This means: if you start one pointer at the head and another at the meeting point, and move both 1 step at a time, they meet at the loop start.

The Complete Algorithm

```
public void detectAndRemoveLoop(ListNode head) { if (head == null || head.next == null) return; ListNode slow = head, fast = head; // Step 1: Detect cycle boolean hasCycle = false; while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; if (slow == fast) { hasCycle = true; break; } } if (!hasCycle) return; // Step 2: Find the start of the loop slow = head; // Edge case: loop starts at head (slow == fast already at head) if (slow == fast) { while (fast.next != slow) { fast = fast.next; } } else { while (slow.next != fast.next) { slow = slow.next; fast = fast.next; } } // Step 3: Remove the loop (fast is now just before the loop start) fast.next = null; }
```

Dry Run

List: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 3$ (cycle back to node 3)
Detection: Step 1: `slow=2, fast=3` Step 2: `slow=3, fast=5` Step 3: `slow=4, fast=4` → MATCH! Cycle detected. Find loop start: Reset `slow` to head (1), `fast` stays at 4
Move both 1 step: `slow=2, fast=5` Move both 1 step: `slow=3, fast=3` → MATCH! Loop starts at node 3. Remove: find the node whose `.next` is 3 (that's node 5), set `5.next = null`.

Complexity

- **Time:** $O(n)$

- **Space:** $O(1)$ — no extra data structures

Summary Cheat Sheet for Quick Revision

Problem	Core Technique	Time	Space
Max Consecutive 1s with K Flips	Sliding Window	$O(n)$	$O(1)$
Search in Rotated Sorted Array	Modified Binary Search	$O(\log n)$	$O(1)$
Zig-Zag Level Order	BFS + LinkedList addFirst	$O(n)$	$O(w)$
Validate BST	DFS with min/max range	$O(n)$	$O(h)$
Reverse in K Groups	Iterative pointer manipulation	$O(n)$	$O(1)$
Detect & Remove Loop	Floyd's Cycle Detection	$O(n)$	$O(1)$